

LE COMUNICAZIONI IN AMBIENTE DISTRIBUITO

In questo capitolo, discuteremo diverse strategie che permettono alle applicazioni distribuite di comunicare. Nel Capitolo 4, abbiamo già fornito una panoramica dei socket e del metodo di chiamata remoto (RMI); in questo capitolo, daremo ulteriori esempi di programmi socket scritti in Java ed anche esempi supplementari di programmi scritti mediante RMI. Inoltre, tratteremo di CORBA che permette alle applicazioni distribuite (e possibilmente eterogenee) di comunicare tra loro.

1 Socket

Un **socket** è un punto di comunicazione ed è identificato da un indirizzo IP concatenato con un numero di porta. Una coppia di processi di comunicazione che comunica in rete impiega una coppia di socket: una per ogni processo; tale coppia definisce la connessione fra le coppie di processi comunicanti.

Nel Capitolo 4 abbiamo studiato un server della data che ascolta una porta specificata. Quando riceve una connessione da un client, restituisce al client la data e l'ora corrente, chiude la connessione e riprende l'ascolto in attesa di altre richieste dai client. Questo server è scritto usando un singolo thread per prestare servizio ad ogni client e richiede solo la consegna dell'ora del giorno; la durata della connessione è breve in modo da permettere al server di prestare servizio a molti client concorrenti senza notevoli ritardi.

Si consideri, tuttavia, la situazione in cui prestare servizio ad un client richiede di mantenere una connessione con esso; come esempio, si consideri un **server dell'eco** (echo server) che semplicemente “ripete l'eco” del testo esatto che riceve dal client e lo ritrasmette al client che lo ha spedito. Una volta stabilita la connessione fra il client ed il server dell'eco, il server aspetta l'immissione del testo dal client, ritornando l'eco di qualsiasi testo ricevuto dal client. Il server può ascoltare le connessioni successive del client solo quando si chiude la connessione con il client attuale.

Si pensi di voler implementare un server dell'eco mediante un singolo thread come è mostrato di seguito:

```
while (true) {  
    // in ascolto per le connessioni  
    Socket client = sock.accept ();  
  
    /**  
    • Ripete l'eco di ciò che è ritornato dal client.  
    • Riprenderemo l'ascolto ai successivi  
    • collegamenti solo quando il client corrente  
    • indica che desidera terminare la connessione.  
    */  
}
```

In questa situazione, siamo in grado di servire un solo client alla volta poichè il singolo thread è dedicato alla connessione corrente; i client successivi devono aspettare che il collegamento corrente termini prima di venire serviti. Un tale progetto non fornisce un server reattivo se si desidera servire rapidamente le connessioni provenienti dal client.

Nel Capitolo 5 abbiamo introdotto i thread e ciò sembra indirizzare il problema di come rendere il server più reattivo. Potremmo costruire un server a più thread e servire ogni connessione di un client in un thread separato, che chiameremo `EchoThread`. Un tale progetto appare come il seguente:

```
while (true) {  
    //in ascolto per le connessioni  
    Socket client = sock.accept ();  
    //passa il socket ad un thread separato  
    /e riprende l'ascolto per altre richieste  
    EchoThread = new EchoThread (client);  
    echo.start ();  
}
```

Questa soluzione permette alle richieste del client di essere servite rapidamente in un nuovo thread o mediante un pool di thread. Tuttavia, ciò può condurre ad un progetto costoso quando si considera il tipico client dell'eco: per gran parte del tempo può essere inattivo e trasmettere solo raramente il testo al server. Pertanto, useremo un thread separato nel server per mantenere una connessione, che spesso può essere inattiva, con ogni client.

Il progetto di tali server non è **scalabile**. Un servizio si definisce scalabile se fornisce un incremento di rendimento, corrispondente ad un aumento nel carico del servizio. Inoltre, una volta che il rendimento raggiunge un punto di saturazione (cioè, non si può ottenere un tasso di rendimento più elevato), qualsiasi carico addizionale non dovrà far diminuire il rendimento. A causa del costo di utilizzazione di un thread separato per mantenere lo stato di ogni connessione del client, il server può permettere solo un certo numero di collegamenti prima del sovraccarico e saturare il server, causando quindi un ritardo maggiore nel rispondere ad ogni client.

Sebbene stiamo usando, come esempio, una semplice applicazione dell'eco, l'uso di un thread separato su un server, per mantenere lo stato di ogni connessione del client, può impedire la scalabilità di molte altre applicazioni client-server quali web e file server.

Nel Capitolo 13 abbiamo introdotto l'I/O non bloccante, che permette all'input ed all'output di operare senza bloccare le chiamate di sistema. Si pensi a come l'I/O è normalmente realizzato: avviene una chiamata di sistema (cioè, `read()`); l'applicazione si blocca finché i dati richiesti (nella loro interezza) non sono disponibili. Con l'I/O non bloccante, si esegue una chiamata di sistema e immediatamente ritorna con tanti dati quanti sono attualmente disponibili. È possibile che nessun dato sia disponibile, richiedendo quindi che l'applicazione sia codificata per eseguire continuamente la chiamata di sistema non bloccante, finché non legge qualsiasi dato disponibile. Tuttavia, il sistema può anche fornire un'indicazione quando almeno qualche dato sia disponibile. Mediante tali conoscenze, un'applicazione può essere codificata alternativamente per eseguire solo la chiamata di sistema non bloccante nel caso ci sia un'indicazione che alcuni dati sono disponibili.

Possiamo progettare server più scalabili usando l'I/O non bloccante, invece di utilizzare un thread separato per la connessione di ciascun client. I progetti di server di questo tipo permettono spesso più connessioni concorrenti del client poichè non si richiede l'overhead di un thread per mantenere lo stato della connessione di ogni client, ma lo stato di ogni connessione è mantenuto dal server e, un cambiamento di stato, è innescato solo da un evento I/O. Inoltre, per minimizzare l'overhead richiesto per mantenere la connessione di ogni client, si deve minimizzare il tempo di risposta quando si risponde ad ogni client.

Questo non significa che ciò che è stato esposto nella presentazione dei thread non fosse utile, ma i moderni server scalabili, quali i web e i chat server e i server di messaggi istantanei, si basano tipicamente sia sui thread che su I/O non bloccante per prestare servizio a un gran numero di connessioni dei client con un tempo minimo di latenza della risposta. Nel paragrafo successivo presenteremo le caratteristiche di I/O non bloccanti, presenti in Java.

1.1 I/O non bloccante in Java

In questo paragrafo, presenteremo il server dell'eco in Java mediante I/O non bloccante. In questo programma, che è mostrato in Figura 1, ci sono parecchie nuove funzionalità, tra cui:

1. insiemi di caratteri,
2. buffer,
3. canali,
4. selettori.

Gli **insiemi di caratteri (character sets)** sono rappresentazioni dei caratteri, esempi includono il carattere romano "A" e il carattere greco λ. Ad ogni carattere, nell'insieme, è tipicamente assegnato un valore numerico e ci sono molte rappresentazioni differenti dei caratteri nei corrispondenti valori numerici: ASCII e Unicode sono due esempi di questo tipo. Poiché l'I/O in rete, coinvolge tipicamente le trasmissioni di flussi di byte, il valore numerico di ogni carattere deve essere **codificato e decodificato** in entrambe le direzioni da una sequenza di byte.

Un **buffer** è una sequenza lineare di elementi che contiene tipi primitivi (cioè `byte`, `char`, `int`, ecc.). Ad un buffer è associata una *capacità* e una *posizione*. La capacità si riferisce al numero di elementi che il buffer può contenere e viene stabilita quando il buffer è allocato per la prima volta e non può cambiare. La posizione di un buffer si riferisce all'indice logico dell'elemento successivo da leggere o da scrivere. Dopo aver disposto un certo numero di elementi nel buffer, la lettura dei dati dal buffer richiede, dapprima, di porre la posizione di nuovo a zero.

I **canali** sono come dei condotti per le operazioni di I/O dai file, dai dispositivi hardware e dai socket. Tipicamente, i canali ed i buffer lavorano in coppia in modo che il contenuto di un buffer possa essere scritto o letto da un canale.

I **selettori** forniscono il meccanismo per realizzare in Java I/O non bloccante. I selettori sono usati per far funzionare in modo concorrente molti canali I/O. Invece di bloccarsi, mentre attende l'input da un canale, ignorando, quindi, i canali rimanenti, un selettore può aspettare simultaneamente che l'I/O sia pronto da un qualsiasi numero di canali. Quando l'I/O è disponibile su uno o più canali, il selettore indica poi quale canale(i) è pronto e, a questo punto, I/O si può eseguire sui canali disponibili.

Prima di addentrarci nei dettagli di I/O non bloccante in Java, illustriamo dapprima come far funzionare il server dell'eco di Figura 1. Per gli scopi di questa discussione, supporremo che il server stia funzionando in un localhost con indirizzo IP 127.0.0.1.

Il server viene avviato con il comando

```
Java NBEchoServer
```

ed ora è in ascolto delle richieste di connessione del cliente alla porta 6007. Un client può collegarsi al server usando `telnet`, mediante il comando

```
telnet 127.0.0.1 6007
```

Ora, è stata stabilita una connessione fra il client e il server dell'eco. Quando il client immette il testo, esso è inviato al server con eco di nuovo al client. Se il client desidera terminare la connessione, immette un punto.

Dopo aver presentato questo materiale necessario, ora siamo pronti a presentare i particolari della Figura 1. Dapprima esamineremo il costruttore di Figura 2.

L'insieme di caratteri da utilizzare è **ISO-8859-1** che è adatto alla maggior parte dei caratteri occidentali; poi, creeremo un insieme codificatore e decodificatore per codificare e decodificare, fra i flussi di byte, e l'insieme di caratteri specificato.

```

import java.io.*;
import java.nio.*;
import java.nio.channels.*;
import java.net.*;
import java.util.*;
import java.nio.charset.*;

public class NBEchoServer
{
    private static final int PORT = 6007;
    private Charset charset;
    private CharsetDecoder decoder;
    private CharsetEncoder encoder;
    private ByteBuffer;
    private ServerSocketChannel server;
    private ServerSocket sock;
    private Selector acceptSelector;

    public NBEchoServer() throws IOException {
        //Figura 2
    }
    public void startServer() throws IOException {
        //Figura 3
    }
    public static void main(String[] args) {
        try {
            (new NBEchoServer()).startServer ();
        }
        catch (IOException ioe) {
            System.err.println (ioe);
        }
    }
}

```

Figura 1. Il server dell'eco che usa un I/O non bloccante.

```
public NBEchoServer() throws IOException {
    charset = Charset.forName (" ISO-8859-1");
    Charsetdecoder decoder = charset.newDecoder();
    Charsetencoder encoder = charset.newEncoder();
    ByteBuffer buffer = ByteBuffer.allocate (256);
    server = ServerSocketChannel.open();
    sock = server.socket();
    sock.bind (new InetSocketAddress (PORT));
    server.configureBlocking (false);
    acceptSelector = Selector.open();
    server.register (acceptSelector, electionKey.OP_ACCEPT);
}
```

Figura 2. Costruttore di NBEchoServer.

L'istruzione

```
buffer = ByteBuffer.allocate (256);
```

crea un buffer che contiene elementi di byte con capacità di 256 byte. Poi, si crea un `ServerSocketChannel` con l'istruzione

```
server = ServerSocketChannel.open();
```

Il socket associato con questo canale viene poi ritrovato, invocando il metodo `socket()` sul `ServerSocketChannel`. Inizialmente il `ServerSocket` non è collegato, si usa il metodo `bind()` per collegare questo socket alla porta di default. L'istruzione `server.configureBlocking (false)` configura il canale del socket come non bloccante.

Poi apriamo un `selector` che useremo per gestire canali multipli mediante il metodo `open()` che è un metodo static nella classe `Selector`. I canali, che devono essere gestiti dal selettore, devono prima registrarsi con il selettore ed includere il tipo di disponibilità che il selettore si aspetterà. L'istruzione

```
server.register (acceptSelector,  
                SelectionKey.OP_ACCEPT);
```

registra il canale socket `server`, che indica il tipo di disponibilità che il selettore può aspettarsi da questo canale (`SelectionKey.OP_ACCEPT`) per una richiesta di connessione da parte di un client.

Una volta creata un'istanza `NBEchoServer`, invochiamo il metodo `startServer()` di Figura 3. Il server dell'eco dapprima invoca il metodo `select()` del selettore. Anche nel caso di I/O non bloccante, la chiamata a `select()` si bloccherà finché almeno uno dei canali registrati non è pronto.

Si ricordi che inizialmente abbiamo registrato solo il `server` del canale del socket; l'unica disponibilità che possiamo inizialmente aspettarci da questo selettore è la connessione delle richieste dai client.

Dopo il ritorno da `select()`, estraiamo poi un insieme di `SelectionKeys` con la chiamata a `acceptSelector.selectedKeys()`; questo gruppo di chiavi rappresenta completamente l'I/O che è disponibile in tutti i canali registrati con il selettore. Si può verificare il tipo di disponibilità di ogni chiave di selezione disponibile. Per gli scopi del nostro server dell'eco, ogni chiave di selezione può essere o (1) una richiesta di connessione (`isAcceptable()` ritorna `true`), o (2) la segnalazione che sono pronti dei dati per essere letti dal canale (`isReadable()` ritorna `true`).

Consideriamo il caso in cui il nostro server dell'eco abbia ricevuto la sua prima richiesta di connessione: in questo caso, chiamiamo il metodo `accept()` del canale del server socket per accettarlo. Normalmente `accept()` è una chiamata bloccante, ma siccome abbiamo configurato il canale come non bloccante, `accept()` ritornerà immediatamente, presumibilmente col valore `null`. Tuttavia, *sappiamo* che `accept()` restituirà una connessione del socket poiché la disponibilità del canale indicava una richiesta di connessione; cioè `isAcceptable()` per `SelectionKey` ha ritornato `true`. Recuperiamo poi il canale associato con la connessione del socket del client mediante l'istruzione `client = server.accept()` e lo configuriamo come non bloccante; infine registriamo il canale del client con il selettore. In questa situazione, questo canale registrerà la propria disponibilità come `SelectionKey.OP_READ`, indicando al selettore che il tipo di disponibilità che può aspettarsi da questo canale, è che i dati siano disponibili per

la lettura. Siccome abbiamo configurato il canale, associato con la connessione del client, come non bloccante, le letture successive da questo canale restituiranno qualsiasi dato disponibile nel canale. Tuttavia leggeremo soltanto da questo canale, solo quando il selettore indica la disponibilità del canale di I/O.

Ora abbiamo due canali registrati con il selettore: (1) il canale del socket del server che aspetta ulteriori connessioni del client e (2) il canale associato con la prima connessione del client. Chiamate successive al metodo `select()` del selettore possono indicare la disponibilità su entrambi i canali.

Se la chiave di selezione è leggibile (`isReadable()` ritorna `true`), ciò indica che un client ha scritto dei dati nel socket. Il server dell'eco deve leggere questi dati dal canale disponibile e inviare di nuovo l'eco al client. Dapprima recuperiamo il canale con i dati disponibili con l'istruzione:

```
clientChannel = (SocketChannel) key.channel();
```

Si ricordi la nostra discussione precedente in cui i canali e i buffer lavoravano in coppia; i dati disponibili da un canale devono essere letti in un buffer. Abbiamo letto i dati del canale nel buffer con l'istruzione `clientChannel.read(buffer)`. Tuttavia, dopo aver letto il contenuto del canale nel buffer, la posizione del buffer è regolata sull'elemento successivo da leggere nel buffer. Per estrarre il contenuto del buffer, si deve riportare di nuovo a zero la posizione del buffer con l'istruzione `buffer.flip()`; decodifichiamo poi il contenuto del buffer in `String object` in modo da poter stabilire se il client desidera terminare la connessione (immettendo un punto ``.``). Se non si desidera terminare la connessione, scriviamo (cioè echo) di nuovo il contenuto del buffer del canale del client con l'istruzione `clientChannel.write(buffer)`. È importante, tuttavia, notare che si deve eseguire di nuovo `flip()` nel buffer prima di riscrivere nel canale; ciò è necessario perché la posizione del buffer è stata modificata al momento della decodifica del buffer in `String object`. Prima di riprendere l'esame del selettore, si deve, in primo luogo, ripulire il buffer per prepararlo alle nuove letture.

Da ultimo, si noti come si maneggia la situazione nel caso che il client indichi che desidera terminare il collegamento (immettendo un punto ``.``) o se il client ha chiuso il proprio canale; questa situazione è rilevata dal server quando il metodo `read()` restituisce `-1` dal canale. In entrambe le situazioni, si deve, in primo luogo, cancellare con il selettore la registrazione del canale invocando l'istruzione `key.cancel()`; seguito dalla chiusura del canale nel server.

```

Public void startServer() throws IOException {
    while (true) {
        acceptSelector.select();
        Set keys = acceptSelector.selectedKeys();

        for (Iterator itr = keys.iterator(); itr.hasNext();) {
            SelectionKey key =(SelectionKey) itr.next();
            itr.remove ();
            if (key.isAcceptable()) {
                socketchannel client = server.accept();
                client.configureBlocking (false);
                SelectionKey clientKey = client.register
                    (acceptSelector, SelectionKey.OP_READ);
            }
            else if (key.isReadable ()) {
                SocketChannel clientChannel =
                    (SocketChannel) key.channel ();
                if (clientChannel.read (buffer) == -1) {
                    key.cancel ();
                    clientChannel.close ();
                }
                else {
                    buffer.flip ();
                    String message
                        = decoder.decode (buffer).toString ();
                    if (message.trim () .equals (".")) {
                        clientChannel.write (encoder.encode
                            CharBuffer.wrap (" BYE"));
                        key.cancel ();
                        clientChannel.close ();
                    }
                    else {
                        buffer.flip (); clientChannel.write (buffer);
                    }
                }
            }
            buffer.clear ();
        }
    }
}

```

Figura 3. Metodo startServer () per il server dell'eco non bloccante.

2 Socket UDP

La nostra trattazione dei socket ha finora considerato solo socket orientati alla connessione TCP. Quando diciamo che i socket TCP sono **orientati alla connessione** (connection-oriented), ci riferiamo alla connessione tra una coppia di socket; questo collegamento si comporta in modo simile ad una chiamata telefonica tra due persone che chiameremo Bob e Alice. Supponiamo che Bob desideri chiamare Alice; egli compone il numero di telefono e aspetta che Alice stacchi la cornetta; in tal modo si stabilisce una connessione tra i due che termina solo quando entrambi riappendono. Abbiamo riprova della connessione nei nostri programmi coi socket TCP in Java con `InputStream` e `OutputStream`, disponibili con un socket.

Inoltre, i socket TCP sono considerati **affidabili** (reliable) intendendo che tutti i dati saranno trasmessi in modo affidabile attraverso il socket. TCP fornisce affidabilità tramite il riconoscimento di corretta ricezione dei dati: il ricevente invia un riconoscimento al mittente. TCP mantiene anche un **controllo di flusso** (flow control) fra gli host in comunicazione che impedisce ad un trasmettitore veloce di sovraccaricare con i dati un ricevente lento. Da ultimo, TCP fornisce un **controllo di congestione** (congestion control) che impedisce al mittente di trasmettere troppi dati in rete, saturando quindi le prestazioni generali della rete. La distinzione fondamentale fra controllo di flusso e controllo di congestione è che il controllo di flusso è un problema fra mittente e ricevente, il controllo di congestione è un problema di tutta la rete.

TCP è usato nella maggioranza dei protocolli Internet tra cui HTTP, FTP, SMTP e Telnet; tali protocolli richiedono l'affidabilità fornita dai socket TCP. Per esempio, i client web usano HTTP per trasferimenti attendibili di file da un server web, gli utenti di posta elettronica richiedono che SMTP sia affidabile nell'invio e nella ricezione di e-mail; tuttavia, l'affidabilità, il flusso ed il controllo di congestione fornito da TCP, comporta un aumento di overhead che, tipicamente, si rileva nella consegna rallentata dei dati rispetto ad un meccanismo di consegna che non fornisce alcuna garanzia di affidabilità.

I socket UDP sono privi di connessione e non affidabili: non c'è alcuna connessione stabilita fra gli host di comunicazione e non c'è alcuna garanzia che i dati siano inoltrati in modo affidabile attraverso il socket. Siccome UDP non presenta l'overhead di TCP, i dati inviati tramite UDP arrivano più rapidamente di quelli tramite TCP. Un protocollo inaffidabile come UDP si rivolge ad applicazioni che possono tollerare un certo livello di perdita di pacchetti quali il flusso audio e video, ove pochi pacchetti persi possono non venire notati da chi ne usufruisce.

2.1 Server della data UDP

In questo paragrafo costruiremo un'applicazione che restituisce la data e l'ora corrente per mezzo di socket UDP in modo simile alla soluzione basata su TCP, che abbiamo visto precedentemente nel Capitolo 4. Il server ascolterà alla porta 6013 (scelta arbitrariamente) le richieste del client. Le due classi chiave di Java in uso con UDP sono `DatagramSocket` e `DatagramPacket`. Un `DatagramSocket` si usa per stabilire un socket UDP. A differenza dei socket basati su TCP che fanno distinzione fra i socket del server e quelli del client (`ServerSocket` e `Socket`), si usa un `DatagramSocket` sia per i server che per i client.

Come precedentemente descritto, i socket UDP sono privi di connessione; ciò significa che non c'è alcuna connessione e quindi nessun `InputStream` o `OutputStream` disponibile per la comunicazione. Tutti i dati, incluso l'indirizzo e la porta a cui si sta consegnando, sono incapsulati in un `DatagramPacket` che viene trasportato mediante il `DatagramSocket`. In questo paragrafo, abbiamo usato in precedenza una conversazione telefonica come analogia per i socket TCP orientati alla connessione ove è necessario solo l'indirizzo (ossia il numero di telefono) per stabilire la connessione. Una volta stabilita la connessione, essa può essere usata per trasmettere e ricevere i dati senza ulteriore indirizzamento. L'analogia equivalente per i socket UDP è il sistema

postale in cui la comunicazione è priva di connessione e ha luogo con le lettere. Invece, in un sistema orientato alla connessione ove è necessario un indirizzo, solo durante l'istituzione della connessione, ogni `DatagramPacket` deve contenere l'indirizzo del ricevente.

Una volta stabiliti gli elementi base UDP, diamo un sguardo al client della data come appare in Figura 4.

Il client dapprima stabilisce un `DatagramSocket` con l'istruzione:

```
server = new DatagramSocket();
```

Ciò abilita nel sistema client, un socket che lo collega ad un numero di porta arbitrario. Siccome UDP è inaffidabile, può darsi che il client debba aspettare una risposta dal server che potrà non ricevere mai. Porremo il tempo massimo del socket a 5000 millisecondi (5 secondi) con l'istruzione `theSocket.setSoTimeout(5000)` e se viene superato il tempo massimo del socket, si avrà una `IOException`.

Il client costruirà poi un `DatagramPacket` con i byte dei dati posti a zero, il datagramma includerà pure l'indirizzo IP del server (si userà `localhost` o `127.0.0.1`) e la porta che il server sta ascoltando (6013).

Ciò si realizza con l'istruzione:

```
sendPacket = new DatagramPacket (new byte [0], 0,  
                                server, PORT);
```

Perchè zero byte di dati? Il client sta semplicemente inviando un datagramma vuoto al server che indica che desidera la data e l'ora corrente; il server invierà di nuovo al cliente un `DatagramPacket` con l'ora e la data come propri dati.

Prima di inviare un pacchetto vuoto al server con l'istruzione `theSocket.send(sendPacket)`, il client dovrà dapprima costruire un `DatagramPacket` di dimensione `BUFFER_SIZE` che sarà usato per memorizzare l'ora e la data che riceverà dal server; ciò si realizza con l'istruzione:

```
byte [] buffer = new byte [BUFFER_SIZE];  
receivePacket = new DatagramPacket (buffer, buffer.length);
```

Il client poi consegna il datagramma `sendPacket` al socket e aspetta il risultato tramite `theSocket.receive (receivePacket)`. La risposta dal server sarà incapsulata nel datagramma `receivePacket`; la porzione di dati del `DatagramPacket` è consegnata come un array di byte che il client deve estrarre usando il metodo `getData()` della classe `DatagramPacket`. Il client determinerà anche la lunghezza del campo dei dati con `getLength()` pure nella classe `DatagramPacket`. Mediante i dati e la relativa lunghezza, il client costruisce poi un oggetto `String` dell'ora e della data corrente.

Stabilito il ruolo del client, diamo ora uno sguardo al versante server dell'applicazione, come è mostrato in Figura 5. Il server dapprima stabilisce un `DatagramSocket` e ascolta la porta 6013 in attesa di richieste del client mediante l'istruzione:

```
server = new DatagramSocket (PORT);
```

Il server costruisce poi un `DatagramPacket` da 1 byte per ricevere le richieste del client. Come notato prima, il client non invierà alcun dato nel datagramma che consegnerà al server, in modo che per il server abbia significato costruire anche un datagramma con i byte dei dati posti a zero, al fine di ricevere le richieste del client. Tuttavia, alcune implementazioni della macchina virtuale Java (JVM), richiedono, quando si ricevono i dati, un campo dati contenente almeno 1 byte; di conseguenza, nel testo adotteremo questo metodo conservativo. Il server attende poi le richieste del client mediante l'istruzione:

```
server.receive (receivePacket);
```

```

import java.net.*;
import java.io.*;

public class UDPDateClient {
    public static void main(String[] args) throws IOException {
        final int PORT = 6013;
        final int BUFFER_SIZE = 256;
        DatagramSocket theSocket= null;

        try {
            theSocket =new DatagramSocket();
            theSocket.setSoTimeout (5000);
            InetAddress Server = InetAddress.getByName
                ("127.0.0.1");

            //costruisce un pacchetto di lunghezza zero da
            //trasmettere al server
            DatagramPacket sendPacket =
                new DatagramPacket(new byte[0],0,server,PORT);

            //costruisce un pacchetto per ricevere una risposta
            //dal server
            byte[] buffer = new byte[BUFFER_SIZE];
            DatagramPacket receivePacket =
                new DatagramPacket(buffer, buffer.length);

            //invia il pacchetto al socket
            theSocket.send(sendPacket);

            //aspetta una risposta
            theSocket.receive(receivePacket);

            //estrae la data dal datagramma
            byte[] data = receivePacket.getData();
            int length = receivePacket.getLength();

            String currentDate = new String(data,0,length);
            System.out.println(currentDate);
        }
        catch (IOException ioe) {
            System.err.println (ioe);
        }
        finally {

            if (theSocket != null)
                theSocket.close (); }
    }
}

```

Figura 4. Client della data che usa i socket UDP.

```

import java.net.*;

```

```

import java.io.*;
public class UDPDateServer
{
    public static void main(String[] args) throws IOException {
        final int PORT= 6013;
        DatagramSocket Server = null;

        try {
            //crea un Datagramsocket in ascolto alla porta di
            default
            server = new DatagramSocket (PORT);

            //pacchetto per ricevere datagrammi vuoti
            DatagramPacket receivePacket =
                new DatagramPacket (new byte[1],1);

            while (true) {
                server.receive (receivePacket);

                //ottiene l'indirizzo IP e la porta del mittente
                InetAddress
                remoteAddr=receivePacket.getAddress();
                Int remotePort = receivePacket.getPort();

                //registra la data corrente ed estrae il byte
                equivalente
                String currentDate=(new java.util.Date()).toString();
                byte[] data=currentDate.getBytes("ISO-8859-1");
                DatagramPacket sendPacket = new
                DatagramPacket(data, data.length, remoteAddr,
                remotePort);
                server.send (sendPacket);
            }
        }
        Catch (IOException ioe) {
            System.err.println (ioe);
        }
        finally {
            if (server != null)
                server.close();
        }
    }
}

```

Figura 5. Server della data che usa i socket UDP.

Quando il server riceve una richiesta dal client, risponderà con ora e data corrente. Il server deve conoscere l'indirizzo IP e la porta del cliente a cui inviare questi dati. (Si ricordi che, con i socket orientati alla connessione TCP, questo non era un problema poichè vi erano l'InputStream e l'OutputStream associati con il socket). Il server estrarrà queste informazioni con le istruzioni

```

InetAddress remoteAddr = receivePacket.getAddress();
int remotePort = receivePacket.getPort ();

```

Il server determina la data corrente (come oggetto `String`) e ricava il byte equivalente mediante il metodo `getBytes()` della classe `String`. Ottenuti l'indirizzo, la porta del sistema remoto e la data, come data corrente, il server crea poi un nuovo `DatagramPacket` e lo manda al mittente.

3 Invocazione dei metodi remoti

L'**invocazione dei metodi remoti** (remote method invocation: RMI), è una caratteristica Java che permette ad un thread di invocare un metodo su un oggetto remoto. Gli oggetti si considerano remoti se risiedono in una differente **macchina virtuale Java (JVM)**. Pertanto, l'oggetto remoto può risiedere in una differente JVM nello stesso computer o in un host remoto collegato in rete. Permettendo ad un programma Java di invocare metodi su oggetti remoti, RMI rende possibile agli utenti lo sviluppo di applicazioni Java distribuite in rete.

Si ricordi dal Capitolo 4, che RMI realizza gli oggetti remoti per mezzo di stub e skeleton. Uno stub è un proxy per l'oggetto remoto che risiede nel client. Quando un client invoca un metodo remoto, viene chiamato lo stub per l'oggetto remoto. In questo paragrafo, costruiremo una soluzione distribuita di passaggio dei messaggi, tramite RMI, per il problema produttore-consumatore. Nel Capitolo 5 è stata presentata per la prima volta la soluzione di passaggio di messaggi per il problema produttore-consumatore; tale soluzione distribuita consisterà in un oggetto remoto per i messaggi che saranno passati dal riferimento, permettendo che i thread del produttore e del consumatore invochino in remoto i metodi `send()` e `receive()`.

3.1 Oggetti remoti

Definiamo gli oggetti remoti dichiarando in primo luogo un'interfaccia che specifica i metodi che possono essere invocati in remoto. Nel Capitolo 4 abbiamo definito l'interfaccia `Channel` che specifica i metodi `send()` e `receive()` per il passaggio dei messaggi. Per provvedere agli oggetti remoti, questa interfaccia deve anche estendere quella di `java.rmi.Remote`, che identifica come remoti gli oggetti che implementano tale interfaccia. Inoltre, ogni metodo dichiarato nell'interfaccia, deve lanciare l'eccezione `java.rmi.RemoteException`. Per gli oggetti remoti, forniamo l'interfaccia `RemoteChannel` che è mostrata in Figura 6.

La classe che definisce l'oggetto remoto deve realizzare l'interfaccia `RemoteChannel` (Figura 7). In aggiunta alla definizione dei metodi dell'interfaccia, la classe deve anche estendere `java.rmi.server.UnicastRemoteObject`.

L'estensione di `UnicastRemoteObject` permette la creazione di un singolo oggetto remoto che ascolta le richieste dalla rete mediante lo schema di default RMI dei socket per la comunicazione in rete. Questo oggetto include pure un metodo `main()` che crea una istanza dell'oggetto e la registra nel registro RMI che funziona sul server con il metodo `rebind()`. In questo caso, l'istanza dell'oggetto registra se stessa con il nome `MessageQueue`. Si noti anche che il costruttore della classe `MessageQueue` deve lanciare `RemoteException`, nel caso avvenga un guasto di comunicazione o di rete che impedisce a RMI di esportare l'oggetto remoto.

Si noti che le realizzazioni dei metodi `send()` e `receive()` sono dichiarate come `synchronized` per assicurarne la sicurezza coi thread. Si può anche usare la sincronizzazione dei thread Java, descritta nel Capitolo 7, per controllare un accesso concorrente agli oggetti remoti.

```
import java.rmi.*;
```

```
public interface RemoteChannel
    extends Remote
{
    public abstract void send(Object item) throws
        RemoteException;

    public abstract Object receive() throws
        RemoteException;
}
```

Figura 6 . L'interfaccia RemoteChannel.

```

import java.util.*;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.*;

public class MessageQueue UnicastRemoteObject
    implements RemoteChannel
{
    private Vector queue;

    public MessageQueue() throws RemoteException {
        queue = new Vector ();
    }

    public synchronized void send(Object item)
        throws RemoteException {
        // Figura 8
    }

    public synchronized Object receive()
        throws RemoteException {
        // Figura 8
    }

    public static void main (String args []) {
        try {
            RemoteChannel server = new MessageQueue();

            // Collega l'istanza di questo oggetto al nome
            "MessageQueue"
            Naming.rebind ("MessageQueue", server);
        }
        catch(Exception e) {
            System.err.println (e);
        }
    }
}

```

Figura 7. Realizzazione dell'interfaccia RemoteChannel.

```
// Questa parte realizza send non bloccante
public synchronized void send(Object item)
    throws RemoteException {
    queue.addElement (item);
}

//Questa parte realizza receive non bloccante
public synchronized Object receive()
    throws RemoteException {
    if (queue.size() == 0)
        return null;
    else
        return queue.remove(0);
}
```

Figura 8. Metodi `send()` e `receive()`.

3.2 Accesso all'oggetto remoto

Una volta che un oggetto è registrato sul server, un client (come è mostrato in Figura 9) può ottenere un riferimento proxy a questo oggetto remoto dal registro di RMI che funziona sul server, usando il metodo statico `lookup()` nella classe `Naming`. RMI fornisce uno schema di consultazione basato su URL della forma `rmi://host/objectName`, dove `host` è il nome IP (o indirizzo) del server su cui risiede l'oggetto remoto `objectName`. `objectName` è il nome dell'oggetto remoto specificato dal server nel metodo `rebind()` (in questo caso, `MailBox`). Una volta che il client ha un riferimento proxy all'oggetto remoto, crea thread separati per produttore e consumatore, passando ad ogni thread un riferimento proxy dell'oggetto remoto. I frammenti del codice sorgente per i thread del consumatore e del produttore sono mostrati nelle figure 10 e 11. Si noti che i thread del consumatore e del produttore invocano i metodi `send()` e `receive()` come normali invocazioni di metodi. Tuttavia, poiché questi metodi possono lanciare `java.rmi.RemoteException`, essi devono essere messi in blocchi try-catch, per il resto le applicazioni che invocano metodi remoti sono progettate come se stessero invocando ordinari metodi locali.


```

import java.rmi.*;

public class Factory
{
    public Factory() {
        //oggetto remoto
        RemoteChannel mailBox;

        try {
            mailBox = (RemoteChannel) Naming.lookup
                ("rmi://127.0.0.1/MessageQueue");

            Thread producer=new Thread(new producer(mailBox));
            Thread consumer=new Thread(new consumer (mailBox));

            producer.start();
            consumer.start();
        }
        catch (Exception e) {
            System.err.println (e);
        }
    }

    Public static void main (String args[]) {
        Factory client = new Factory();
    }
}

```

Figura 9. La fabbrica del client.

```

import java.util.*;
class Producer implements Runnable
{
    private RemoteChannel mbox;

    public Producer(RemoteChannel m){
        mbox= m;
    }

    Public void run() {
        Date message;

        while (true) {
            //sonnecchia per un po' di tempo
            SleepUtilities.nap();

            //produci un articolo e immettilo nel buffer
            Message = new Date();
            System.out.println ("Producer produced " +
                message);
            try {
                mbox.send (message);
            }
            catch (java.rmi.RemoteException re) {
                System.err.println(re);
            }
        }
    }
}

```

Figura 10. Thread del produttore.

```

import java.util.*;

Class Consumer implements Runnable
{
    private RemoteChannel mbox;

    public Consumer(RemoteChannel = m) {
        mbox = m;
    }

    public void run() {
        Date message;

        while (true) {
            //sonnecchia per un po' di tempo
            SleepUtilities.nap ();

            //consumo un articolo nel buffer
            try {
                message = (Date) mbox.receive();
                if (message != null)
                    System.out.println ("Consumer consumed "
                        + message);
            }
            catch (java.rmi.RemoteException re) {
                System.err.println (re);
            }
        }
    }
}

```

Figura 11. Thread del consumatore.

3.3 Funzionamento dei programmi

Ora mostriamo i passi necessari per fare funzionare i programmi dell'esempio. Per semplicità, supponiamo che tutti i programmi funzionino sull'host locale. Tuttavia, la comunicazione va ancora considerata remota, perché i programmi del server e del client girano ognuno sulla propria JVM.

1. *Compilare tutti i file sorgente.*
2. *Generare lo stub e lo skeleton.* Lo strumento RMIC è utilizzato per generare i file delle classi stub e skeleton, essi sono generati dall'utente immettendo

```
rmic MessageQueue
```

nella riga di comando; questo crea i file `MessageQueue_Skel.class` e `MessageQueue_Stub.class`. (Se si esegue questo esempio su due differenti computer, bisogna assicurarsi che tutti i file della classe, incluse le classi stub, siano disponibili su ogni computer. Si possono caricare dinamicamente le classi mediante RMI, argomento che va oltre lo scopo di questo testo, ma è trattato nei testi menzionati nelle note bibliografiche.)

3. *Avviare il registro e creare l'oggetto remoto.* Per avviare il registro su piattaforme UNIX, l'utente può scrivere

```
rmiregistry &
```

In Windows, l'utente può scrivere

```
start rmiregistry
```

Questo comando avvia il registro presso il quale si registrerà l'oggetto remoto, successivamente si crea una istanza dell'oggetto remoto con

```
java MessageQueue
```

Questo oggetto remoto si registrerà con il nome `MessageQueue`.

4. *Riferimento all'oggetto remoto.* L'istruzione

```
java factory
```

viene inserita nella riga di comando per dare inizio alla factory del client. Questo programma otterrà un riferimento proxy per l'oggetto remoto `MessageQueue` e creerà i thread per il produttore e il consumatore, passando ad ogni thread il riferimento proxy per l'oggetto remoto. I thread del produttore e del consumatore possono ora invocare i metodi `send()` e `receive()` sull'oggetto remoto.

4 Altri aspetti della comunicazione distribuita

Le comunicazioni distribuite sono un'area dell'elaborazione importante e in continua evoluzione. Di conseguenza, ci sono molti aspetti relativi alle comunicazioni distribuite e molte soluzioni alle sfide comuni che si trovano in quell'ambiente. Questo paragrafo tratta di CORBA, un'alternativa a RMI e in quale modo gli oggetti remoti possono registrarsi con i servizi di RMI e CORBA.

4.1 CORBA

RMI è una tecnologia, nativa di Java, che permette ai thread di invocare metodi sugli oggetti distribuiti; tuttavia, richiede che tutte le applicazioni distribuite siano scritte in Java. Molti sistemi esistenti, che potremmo desiderare distribuiti, sono scritti in C, C++, COBOL, o Ada; nessun protocollo di comunicazione, discusso fino ad ora, fornisce un meccanismo conveniente atto alla comunicazione tra queste applicazioni eterogenee.

Common Object Request Broker Architecture (CORBA) è un **middleware**: uno strato software intermedio, che permette ad applicazioni client-server eterogenee di comunicare. Per esempio, un programma C++ potrebbe usare CORBA per accedere ad un servizio della base di dati scritto in COBOL. CORBA permette alle applicazioni scritte in linguaggi differenti di comunicare usando un **linguaggio di definizione delle interfacce** (interface-definition language: IDL) e un **intermediario per la richiesta degli oggetti** (object request broker: ORB). IDL permette ad un oggetto distribuito, quale una base di dati, di descrivere un'interfaccia per i servizi che fornisce. IDL è un generico linguaggio di programmazione che permette ad un servizio di descrivere se stesso indipendentemente da qualsiasi linguaggio specifico. Per comunicare con un server, un client deve comunicare solo con l'interfaccia specificata da IDL e l'oggetto server realizza l'interfaccia specificata da IDL. ORB forma la spina dorsale di CORBA, permettendo ai client di invocare metodi su oggetti distribuiti ed di accettare i valori di ritorno. Esiste ORB sia per i client che per i server, e un protocollo: il **protocollo di Internet di InterORB** (internet interORB protocol: IIOP), che specifica come ORB può comunicare.

Quando un'applicazione client richiede un riferimento ad un oggetto remoto, è responsabilità del client ORB di trovare l'oggetto remoto nel sistema distribuito. Il client ORB è pure responsabile dell'instradamento delle chiamate remote al server appropriato e di accettare i risultati provenienti dal server. Il server ORB permette al server di registrare nuovi oggetti CORBA ed è anche responsabile di accettare le richieste provenienti dai client ORB per invocare metodi sul server. Il server ORB passa anche i valori di ritorno al client.

Un ambiente CORBA lavora nel modo seguente. Quando un client ha un riferimento ad un oggetto remoto, qualsiasi invocazione dei metodi di questo oggetto avviene attraverso lo stub del client che usa l'ORB del client per comunicare con il server. Quando un server ORB riceve una richiesta di invocazione di un metodo dal client, chiama lo skeleton appropriato, che a sua volta invoca la realizzazione del metodo remoto nel server. I valori di ritorno sono passati attraverso lo stesso percorso; questa situazione è mostrata in Figura 12.

Sotto molti aspetti, ORB si comporta in modo molto simile a RMI, tuttavia RMI è una tecnologia da Java a Java. CORBA fornisce interoperabilità fra client e server eterogenei, permettendo alle applicazioni scritte in linguaggi differenti di comunicare tra loro.

4.2 Registrazione degli oggetti

Sia RMI che CORBA sono **sistemi di oggetti distribuiti** (distributed object systems); gli oggetti sono distribuiti in rete e le applicazioni possono comunicare con gli oggetti distribuiti. Una caratteristica importante di tali sistemi è il **servizio di registrazione degli oggetti** (object-registration service) con il quale un oggetto si registra da solo. Le applicazioni che desiderano usare un oggetto distribuito ottengono un riferimento all'oggetto mediante questo servizio. **Rmiregistry** agisce come servizio di registrazione per RMI. Una volta che la registrazione è iniziata, un oggetto remoto si registra con un nome unico usando il metodo `Naming.rebind()`. I client usano questo nome quando richiedono un riferimento all'oggetto remoto dal server di registrazione. Mediante RMI, un client ottiene un riferimento all'oggetto remoto tramite il metodo `Naming.lookup()`. In un sistema CORBA, ORB sul versante server è responsabile di fornire il servizio di registrazione.

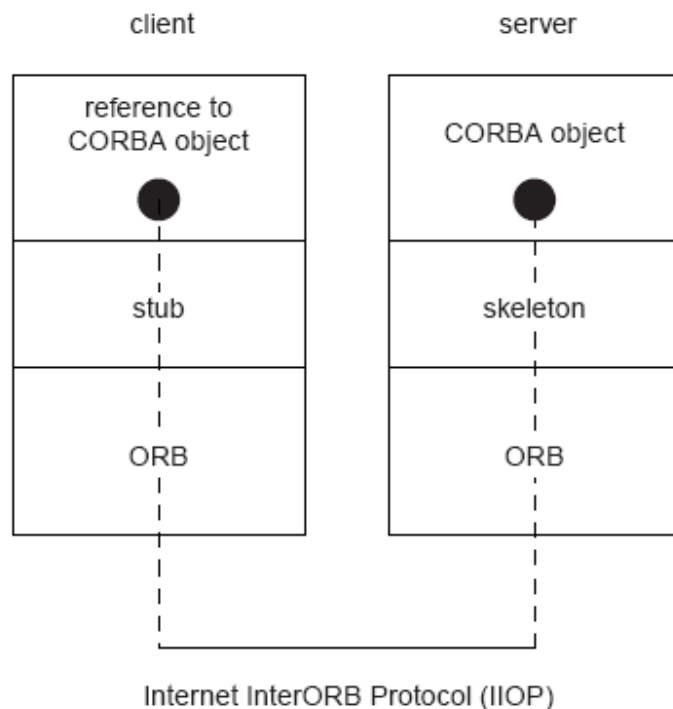


Figura 12. Modello dell'ambiente CORBA.

client

server

reference to CORBA object = riferimento all'oggetto CORBA

CORBA object = oggetto CORBA

stub

skeleton = scheletro

ORB

5 Sommario

In questo capitolo, abbiamo presentato parecchi metodi che permettono alle applicazioni distribuite di comunicare. Il primo, un socket, è definito come un punto finale per la comunicazione. Una connessione fra una coppia di applicazioni consiste in una coppia di socket, ognuno ad una estremità del canale di comunicazione. RPC è un'altra forma di comunicazione distribuita; si presenta RPC quando un processo o un thread chiama una procedura su una applicazione remota. RMI è la versione Java di RPC che permette ad un thread di invocare un metodo su un oggetto remoto come se dovesse invocare un metodo su un oggetto locale. La distinzione principale fra RPC e RMI è che, nel secondo caso, i dati che vengono passati ad una procedura remota si presentano nella forma di una ordinaria struttura dati. RMI permette agli oggetti di venire passati nelle chiamate remote dei metodi. Abbiamo pure discusso di CORBA: una tecnologia che permette

la comunicazione fra applicazioni distribuite scritte in linguaggi differenti. RMI e CORBA sono entrambi sistemi di oggetti distribuiti. Una caratteristica importante di questi sistemi è il servizio di registrazione degli oggetti che registra gli oggetti distribuiti. I client si mettono in contatto con il servizio di registrazione quando desiderano individuare un oggetto distribuito.

I servizi Web sono simili a quelli RPC, ma sono indipendenti dalla piattaforma, dal sistema operativo e dal linguaggio. Sono basati su XML; includono chiamate di procedura remota, ma anche la scoperta della descrizione del servizio e del servizio stesso.

Esercizi

- 1 Scrivere un server basato sul socket “La Ruota della Fortuna” mediante I/O non bloccante. Il programma deve creare un server che ascolta una porta specificata. Quando si riceve una connessione da un client, il server deve rispondere con una vincita casuale scelta dalla propria base di dati delle vincite.
- 2 Scrivere un server dell’eco basata su un socket multithread senza usare I/O non bloccante. Creare un nuovo thread per prestare servizio ad ogni connessione entrante. Confrontare questo progetto con il programma mostrato in Figura 1.
- 3 Progettare un server web HTTP usando I/O non bloccante.
- 4 Progettare una applicazione di un server client di chat mediante I/O non bloccante.
- 5 Descrivere il motivo, per cui prestare servizio a connessioni client in un gruppo di thread, può ancora limitare la scalabilità nel progetto di un server.
- 6 Realizzare l’algoritmo 3 del Capitolo 7 mediante RMI.
- 7 Questo capitolo ha suggerito una soluzione distribuita al problema del buffer limitato nel passaggio dei messaggi. Un'altra soluzione a tale problema ha utilizzato la sincronizzazione Java, come è stato mostrato nel Capitolo 7. Modificare questa seconda soluzione mediante RMI, in modo che la soluzione funzioni in un ambiente distribuito.
- 8 Il problema dei lettori scrittori è stato risolto, nel Capitolo 7, usando la sincronizzazione Java. Modificare quella soluzione in modo che funzioni in un ambiente distribuito mediante RMI.
- 9 Realizzare una soluzione distribuita al problema del pranzo dei filosofi mediante RMI.
- 10 Spiegare differenze e somiglianze fra RMI e CORBA.

Note bibliografiche

Hitchens [2002] presenta un’ampia discussione riguardo I/O non bloccante in Java. Welsh et al.[2001] e Nian-Min et al. [2002] discutono le architetture dei server Internet ad alte prestazioni. Kurose e Ross [2001] e Harold [2000] descrivono la programmazione del socket con particolare riguardo a Java. Grosso [2002] tratta RMI. Farley [1998] si focalizza su RMI e contiene pure un paragrafo che descrive l’uso di CORBA in Java. La homepage di RMI si trova all’indirizzo [<http://www.javasoft.com/products/jdk/rmi>] ed elenca le risorse aggiornate. La homepage di OMG che si trova all’indirizzo [<http://www.omg.org>] è un buon punto di partenza nel Web per informazioni riguardanti CORBA.